

VitalSoC Register Specification (Driver Reference)

Rev 0.1 (draft), 8 October 2026. Audience: embedded firmware engineer writing the drivers. Format modelled on the Infineon PSoC 4 Registers TRM (register map, then per-register detail with access types and defaults).

1. About this document

This is the contract between hardware and firmware. If the RTL and this document disagree, stop and raise it: one of them is wrong.

Status legend

Tag	Meaning
SIM-VERIFIED	RTL exists and its testbench passes (bit-exact against the Python model)
WRITTEN	RTL/firmware written, not yet simulated or run on a board
BOARD-PROVEN	Works on hardware today (the existing bootloader build)
PLANNED	Not integrated yet. The register layout is the intended one and may change
STRETCH	Optional. Do not write drivers until the spec is issued

Access types (same idea as the Infineon TRM)

Code	Meaning
R	Read only. Writes are ignored
W	Write only. Reads return 0 unless stated
RW	Read and write

Code	Meaning
RW1C	Read; writing 1 clears the bit, writing 0 has no effect
RW1S	Set by hardware (used in the HW Access column)
None	Reserved. Read returns 0, write ignored. Always write 0

Global rules

- All register accesses are **naturally aligned 32-bit loads and stores** (`lw` / `sw`). Do not use `lb/lh/sb/sh` on peripherals. In C, use `volatile uint32_t *`.
- Little endian. Registers are 32 bits wide; unused upper bits read as 0.
- Each peripheral has a 4 KB window but decodes only a few address bits, so registers **alias** inside the window. Use only the offsets listed here.
- An access to an unmapped address completes normally and reads back `0xDEADBEEF`. Writes are ignored. Use this as a quick wiring check.
- Reset: the CPU reset button resets the CPU and the peripheral registers. It does **not** clear App RAM, and it does not clear FIR coefficient memory or FIR delay lines (see section 7).
- System clock is 100 MHz on both candidate boards. All timing constants below assume this. Confirm the oscillator on the board you are given.

2. System overview and register mapping

CPU: PicoRV32, RV32IMC, hardware multiply and divide, `rdcycle` counter, interrupts enabled. No atomics, no caches, no MMU.

2.1 Memory map

Base address	Size	Block	Status
<code>0x0000_0000</code>	2 KB	Boot ROM (resident bootloader, read-only)	BOARD-PROVEN
<code>0x1000_0000</code>	64 KB	App RAM (application is uploaded here over UART)	PLANNED (currently 8 KB in the bootloader build)

Base address	Size	Block	Status
0x2000_0000	4 KB	UART	BOARD-PROVEN
0x3000_0000	4 KB	GPIO (LEDs, switches, buttons)	WRITTEN (LED register BOARD-PROVEN)
0x4000_0000	4 KB	Timer	WRITTEN
0x5000_0000	4 KB	FIR accelerator, 4 channels	SIM-VERIFIED
0x6000_0000	4 KB	I2C master	PLANNED
0x7000_0000	4 KB	MI modem	STRETCH

The current RTL still uses an earlier address map (0x1000_xxxx for peripherals) and a different UART. The hardware engineer will move it to the map above. Write the drivers against this table.

2.2 Interrupt map

IRQ line	Source	Type	Status
irq[0]	PicoRV32 internal timer	Reserved, unused	n/a
irq[1]	EBREAK, illegal instruction, ECALL	Reserved	n/a
irq[2]	Bus error / misaligned access	Reserved	n/a
irq[3]	Timer tick	Level (flag AND enable)	WRITTEN
irq[4]	FIR done	Level (flag AND enable)	SIM-VERIFIED
irq[5]	I2C transfer complete	Level	PLANNED
irq[6]	MI modem	Level	STRETCH

The UART has no interrupt. Poll it.

2.3 Reference header (starting point)

```

/* soc_regs.h */
#define BOOTROM_BASE 0x00000000u
#define APPRAM_BASE 0x10000000u
#define APPRAM_SIZE 0x00010000u
#define UART_BASE 0x20000000u
#define GPIO_BASE 0x30000000u
#define TIMER_BASE 0x40000000u
#define FIR_BASE 0x50000000u
#define I2C_BASE 0x60000000u
#define MI_BASE 0x70000000u

#define REG32(base, off) (*(volatile uint32_t *)((base) + (off)))

#define IRQ_TIMER (1u << 3)
#define IRQ_FIR (1u << 4)
#define IRQ_I2C (1u << 5)
#define IRQ_MI (1u << 6)

#define CLK_HZ 100000000u

```

3. Boot ROM, App RAM and the upload interface

This section comes from the bootloader manual. Anything marked "confirm" has not been read from the bootloader source yet.

Boot sequence

1. Reset: the CPU starts at `0x0000_0000` (Boot ROM, `PROGADDR_RESET`).
2. The bootloader waits for an upload. LED register = `0x0001`.
3. Host sends a 4-byte length header, then the raw application bytes. LED = `0x0003` while receiving.
4. Bootloader sends one `'K'` byte as acknowledgement, sets LED = `0xFFFF`, and jumps to `0x1000_0000`.
5. The application runs. Interrupts are masked at reset, so the bootloader is never interrupted.

There is no automatic fallback: after every reset or power cycle the bootloader waits for a new upload, because App RAM is volatile.

Application image requirements

Item	Requirement
Link address	<code>0x1000_0000</code> , length <code>0x1_0000</code> (64 KB)
Entry	First instruction at <code>0x1000_0000</code> is a jump to the C startup
IRQ vector	<code>PROGADDR_IRQ = 0x1000_0010</code> . The handler entry must be placed exactly at offset <code>0x10</code> of the image (<code>start.S</code> already does this: reset jump, then <code>.balign 16</code>)
Output format	Raw binary (<code>objcopy -O binary</code>), not hex
Stack	Application sets its own <code>sp</code> to the top of App RAM (<code>0x1001_0000</code>). The bootloader's own stack must not overlap the uploaded image (confirm)
<code>.bss</code>	Application clears its own <code>.bss</code> at startup

Confirm with the hardware engineer: byte order of the length header; maximum image size accepted; where the bootloader stack lives once App RAM is enlarged.

4. UART (base `0x2000_0000`)

Status: BOARD-PROVEN. This is the UART from the bootloader build. It has no baud-rate register: the rate is fixed in hardware at 115200 baud (`CLKS_PER_BIT = 868` at 100 MHz). Frame format is assumed 8N1 (confirm).

4.1 Register map

Register	Offset	Address
UART_DATA	0x00	<code>0x2000_0000</code>
UART_STATUS	0x04	<code>0x2000_0004</code>

4.2 UART_DATA

Address: `0x2000_0000` . Write sends a byte. Read returns the last received byte.

Bits	Name	SW	HW	Reset	Description
31:8	None	None	None	0	Reserved
7:0	DATA	RW	RW	0	Write: byte to transmit. Read: received byte, valid only when UART_STATUS.RX_VALID = 1

Do not write while TX_BUSY = 1: behaviour is undefined (confirm whether the byte is dropped). Whether a read clears RX_VALID is unconfirmed; the driver should read DATA once per RX_VALID and re-check the flag.

4.3 UART_STATUS

Address: 0x2000_0004 .

Bits	Name	SW	HW	Reset	Description
31:2	None	None	None	0	Reserved
1	RX_VALID	R	RW	0	1: a received byte is waiting in UART_DATA
0	TX_BUSY	R	RW	0	1: transmitter is shifting a byte out

Driver rules

- `uart_putc` : poll TX_BUSY until 0, then write DATA. Translate `\n` to `\r\n` for the terminal.
- `uart_getc_timeout(ms)` : poll RX_VALID with a timeout. Never block forever.
- At 115200 baud one byte takes about 87 μ s. A 30-character line costs roughly 2.6 ms of CPU time, so do not print from the interrupt handler.
- After the upload, the host terminal must be opened on the same COM port at 115200 baud.

5. GPIO (base 0x3000_0000)

Status: WRITTEN. The LED register is BOARD-PROVEN (the bootloader writes it).

5.1 Register map

Register	Offset	Address
GPIO_LED	0x00	0x3000_0000

Register	Offset	Address
GPIO_SW	0x04	0x3000_0004
GPIO_BTN	0x08	0x3000_0008

5.2 GPIO_LED

Bits	Name	SW	HW	Reset	Description
31:16	None	None	None	0	Reserved
15:0	LED	RW	R	0	One bit per LED. 1 = on. Bits above the board's LED count have no effect (see section 11)

The bootloader leaves this register at 0xFFFF when it jumps to the application. The application should write its own value first thing.

Application LED convention: bit 0 toggles on every detected heartbeat; bit 7 = alarm (heart rate out of range). Keep these on bits 0 to 7 so the same firmware works on both boards.

5.3 GPIO_SW

Bits	Name	SW	HW	Reset	Description
31:16	None	None	None	0	Reserved
15:0	SW	R	RW	0	Slide switch state, two-flop synchronised. Bits for switches that do not exist read 0

5.4 GPIO_BTN

Bits	Name	SW	HW	Reset	Description
31:5	None	None	None	0	Reserved
4:0	BTN	R	RW	0	Push-button state, two-flop synchronised. Non-existent buttons read 0. The CPU reset button is not readable

Buttons and switches are not debounced in hardware. Debounce in software (sample on the timer tick and require two equal samples).

6. Timer (base 0x4000_0000)

Status: WRITTEN. A periodic tick generator with one interrupt (`irq[3]`).

6.1 Register map

Register	Offset	Address
TIMER_CTRL	0x00	0x4000_0000
TIMER_PERIOD	0x04	0x4000_0004
TIMER_FLAG	0x08	0x4000_0008
TIMER_TICKS	0x0C	0x4000_000C

6.2 TIMER_CTRL

Bits	Name	SW	HW	Reset	Description
31:2	None	None	None	0	Reserved
1	IRQ_EN	RW	R	0	1: <code>irq[3]</code> is asserted while <code>TIMER_FLAG.TICK = 1</code>
0	ENABLE	RW	R	0	1: counter runs. 0: counter is held at 0 (a set TICK flag stays set)

6.3 TIMER_PERIOD

Bits	Name	SW	HW	Reset	Description
31:0	PERIOD	RW	R	99,999	Reload value. Tick interval = (PERIOD + 1) clock cycles. Default is 1 kHz at 100 MHz

Formula: `PERIOD = CLK_HZ / tick_hz - 1`. If PERIOD is written to a value below the current count, the next tick fires immediately.

6.4 TIMER_FLAG

Bits	Name	SW	HW	Reset	Description
31:1	None	None	None	0	Reserved

Bits	Name	SW	HW	Reset	Description
0	TICK	RW1C	RW1S	0	Set by hardware each time the counter reaches PERIOD. Write 1 to clear

The interrupt handler **must** clear TICK, or `irq[3]` stays high and the CPU re-enters the handler. If hardware sets TICK in the same cycle as a clear, the set wins.

6.5 TIMER_TICKS

Bits	Name	SW	HW	Reset	Description
31:0	TICKS	R	RW	0	Free count of ticks since reset. Not cleared by ENABLE = 0

Recommended use: configure 1 kHz. Derive the 250 Hz ECG sample clock by dividing by 4 in the handler, and the 100 Hz PPG clock by dividing by 10.

7. FIR accelerator (base `0x5000_0000`)

Status: SIM-VERIFIED (core bit-exact on all four channels, interleaved and saturating; register wrapper tested for polling, IRQ and start-while-busy). Not yet run on a board.

7.1 What it does

Four independent FIR filter channels share one multiplier. Each channel has its own 64-tap coefficient bank and its own delay line. For each input sample the hardware computes

$$y[n] = \text{saturate16}(\text{sum over } k = 0..63 \text{ of } \text{coef}[ch][k] * x[ch][n-k]) \gg 15$$

with `x` and `y` signed 16-bit, coefficients signed 16-bit in Q1.15, and a 40-bit internal accumulator. The right shift is arithmetic. One sample takes **about 67 clock cycles** (TAPS + 3) from START accepted to DONE, which is 0.67 μ s at 100 MHz.

The C function `sw_fir_step()` in `dsp.h` is a bit-exact software twin. Use it as the golden reference in every test and in the speed-up benchmark. Its 32-bit accumulator is fine for ECG-sized input but can overflow on full-scale input; use a 64-bit accumulator if you test with full-scale signals.

7.2 Register map

Register	Offset	Address
FIR_CTRL	0x00	0x5000_0000
FIR_STATUS	0x04	0x5000_0004
FIR_DATA_IN	0x08	0x5000_0008
FIR_DATA_OUT	0x0C	0x5000_000C
FIR_COEF_ADDR	0x10	0x5000_0010
FIR_COEF_DATA	0x14	0x5000_0014

7.3 FIR_CTRL

Bits	Name	SW	HW	Reset	Description
31:10	None	None	None	0	Reserved
9:8	CHANNEL	RW	R	0	Channel to process, 0 to 3
7:2	None	None	None	0	Reserved
1	IRQ_EN	RW	R	0	1: <code>irq[4]</code> is asserted while <code>FIR_STATUS.DONE = 1</code>
0	START	W	RW	0	Write 1 to start one filter step. Self-clearing, always reads 0. Ignored while <code>BUSY = 1</code> . Accepting a start clears <code>DONE</code>

A write to `FIR_CTRL` always updates `CHANNEL` and `IRQ_EN`, even when the `START` bit is ignored. A start takes the channel from the same write, so set `CHANNEL` and `START` in one store.

7.4 FIR_STATUS

Bits	Name	SW	HW	Reset	Description
31:2	None	None	None	0	Reserved
1	DONE	RW1C	RW1S	0	Sticky. Set when a step finishes. Write 1 to clear. Cleared automatically when a new start is accepted
0	BUSY	R	RW	0	1: a step is in progress

Use DONE, not BUSY, to detect completion. BUSY rises one or two clocks after the START write, so reading it as 0 immediately afterwards does not mean the step finished.

7.5 FIR_DATA_IN

Bits	Name	SW	HW	Reset	Description
31:16	None	None	None	0	Ignored on write
15:0	DIN	W	R	0	Signed 16-bit input sample. Write it before the START write. Reads return 0

7.6 FIR_DATA_OUT

Bits	Name	SW	HW	Reset	Description
31:0	DOUT	R	RW	0	Last filtered sample, sign-extended to 32 bits. Valid after DONE. Reading does not clear DONE

7.7 FIR_COEF_ADDR

Bits	Name	SW	HW	Reset	Description
31:8	None	None	None	0	Reserved
7:6	CH	W	R	0	Channel whose bank is being written
5:0	TAP	W	R	0	Tap index 0 to 63 (tap 0 multiplies the newest sample)

Reads return 0. Linear address = `channel * 64 + tap`.

7.8 FIR_COEF_DATA

Bits	Name	SW	HW	Reset	Description
15:0	COEF	W	R	n/a	Signed Q1.15 coefficient. The write itself triggers the store at FIR_COEF_ADDR. Reads return 0

7.9 Reset behaviour (important for the bootloader flow)

The CPU reset clears the FIR registers (CTRL, STATUS, IRQ enable) but does **not** clear the coefficient memory, the delay lines or the circular pointers. After pressing reset and uploading a new application, the previous run's coefficients and filter history are still there. The firmware init must therefore always:

1. Load all four coefficient banks.
2. Flush every channel by feeding 64 zero samples, so no old history leaks into the output.

The flush also hides the stale pointer state, because 64 zeros fill the whole delay line.

7.10 Programming sequences

Init (once per application start)

1. For ch in 0..3, for tap in 0..63: write `FIR_COEF_ADDR = (ch << 6) | tap`, then `FIR_COEF_DATA = coef[ch][tap]`.
2. For ch in 0..3, 64 times: run one step with input 0.

One sample, polling

1. `FIR_DATA_IN = x`
2. `FIR_CTRL = (ch << 8) | 1`
3. Loop until `FIR_STATUS & 2`
4. `y = (int16_t)FIR_DATA_OUT`
5. `FIR_STATUS = 2` (clear DONE)

One sample, interrupt driven

1. `FIR_DATA_IN = x`, then `FIR_CTRL = (ch << 8) | 2 | 1` (IRQ_EN and START).
2. CPU continues with other work.
3. In the `irq[4]` handler: read `FIR_DATA_OUT`, then write `FIR_STATUS = 2`. If DONE is not cleared, the interrupt re-fires.

Rules

- Do not write coefficients for a channel while BUSY = 1: a mid-step write corrupts that output sample.
- One step at a time. Do not start a new step until DONE has been seen and cleared.
- Only use full-word stores.

7.11 Default channel plan

The coefficient banks come from `fir_golden.py` and are exported to `coeffs.h`. Treat `coeffs.h` as the source of truth; this table is a summary.

Channel	Signal	Filter	Sample rate
0	ECG (QRS detection)	Band-pass 5 to 25 Hz	250 Hz
1	PPG red	Low-pass 4 Hz	100 Hz
2	PPG IR	Low-pass 4 Hz	100 Hz
3	ECG (spare, used by the self-test)	Low-pass 40 Hz	250 Hz

The filters have a group delay of about 31.5 samples. Remove DC from PPG in firmware before computing the red/IR ratio.

8. I2C master (base `0x6000_0000`)

Status: PLANNED. The IP is the OpenCores `i2c_master_top` behind a bus bridge, so the layout below is the standard OpenCores register set with each register on a 4-byte offset and data in bits 7:0 (bits 31:8 read 0). Confirm offsets with the hardware engineer once the bridge exists.

8.1 Register map

Register	Offset	Address	Access
I2C_PRERLO	0x00	<code>0x6000_0000</code>	RW
I2C_PRERHI	0x04	<code>0x6000_0004</code>	RW
I2C_CTR	0x08	<code>0x6000_0008</code>	RW
I2C_TXR / I2C_RXR	0x0C	<code>0x6000_000C</code>	W (TXR) / R (RXR)
I2C_CR / I2C_SR	0x10	<code>0x6000_0010</code>	W (CR) / R (SR)

8.2 Prescaler (I2C_PRERLO, I2C_PRERHI)

$\text{prescale} = \text{CLK_HZ} / (5 * \text{SCL_HZ}) - 1$. For 100 MHz and 100 kHz this is 199 (`PRERLO = 0xC7`, `PRERHI = 0x00`). Write it while the core is disabled (`CTR.EN = 0`). Reset value is `0xFFFF`.

8.3 I2C_CTR

Bits	Name	SW	Reset	Description
7	EN	RW	0	1: core enabled
6	IEN	RW	0	1: interrupt enabled (<code>irq[5]</code>)
5:0	None	None	0	Reserved

8.4 I2C_TXR and I2C_RXR

Register	Bits	Description
TXR (write)	7:1	Slave address (7-bit) or data byte
TXR (write)	0	When sending an address: 0 = write to slave, 1 = read from slave. When sending data: data bit 0
RXR (read)	7:0	Last byte received from the slave

8.5 I2C_CR (write only)

Bits	Name	Description
7	STA	Generate (repeated) start condition
6	STO	Generate stop condition
5	RD	Read from slave
4	WR	Write to slave
3	ACK	When acting as receiver: 0 = send ACK, 1 = send NACK (use NACK on the last byte)

Bits	Name	Description
2:1	None	Reserved
0	IACK	Write 1 to clear the interrupt flag

CR bits STA, STO, RD, WR and IACK are self-clearing.

8.6 I2C_SR (read only)

Bits	Name	Description
7	RXACK	1: slave did not acknowledge (NACK). 0: ACK received
6	BUSY	1: bus busy (after a start, until a stop)
5	AL	1: arbitration lost
4:2	None	Reserved
1	TIP	1: transfer in progress. Wait for 0 after every command
0	IF	Interrupt flag, set at the end of each byte transfer. Clear with CR.IACK

8.7 Read one register from a slave (pattern for all sensor reads)

Example reads MAX30102 PART_ID (register `0xFF`, expected value `0x15`). The slave address is `0x57`, so the address byte is `0xAE` for write and `0xAF` for read.

1. `TXR = 0xAE`; `CR = STA | WR`; wait `TIP = 0`; check `RXACK = 0`.
2. `TXR = 0xFF`; `CR = WR`; wait `TIP = 0`; check `RXACK = 0`.
3. `TXR = 0xAF`; `CR = STA | WR` (repeated start); wait `TIP = 0`; check `RXACK = 0`.
4. `CR = RD | ACK | STO` (read one byte, send NACK, then stop); wait `TIP = 0`.
5. Read `RXR`.

For a burst read, repeat `CR = RD` (`ACK = 0`) for every byte except the last, which uses `RD | ACK | STO`.

Driver rules: always put a timeout on every TIP wait; on RXACK = 1 or AL = 1, send a stop, reinitialise the core, and return an error code. Bring up the driver by polling. Use the interrupt only after polling works.

8.8 MAX30102 reference (verify against the datasheet before use)

Register	Address	Notes
INT_STATUS_1	0x00	Read to clear
INT_ENABLE_1	0x02	
FIFO_WR_PTR	0x04	
OVF_COUNTER	0x05	
FIFO_RD_PTR	0x06	
FIFO_DATA	0x07	Burst-read: the register address does not auto-increment
FIFO_CONFIG	0x08	Sample averaging, rollover, almost-full level
MODE_CONFIG	0x09	0x40 = reset. 0x03 = SpO2 mode (red + IR)
SPO2_CONFIG	0x0A	Suggested start: 0x27 (4096 nA range, 100 sps, 411 μ s pulse)
LED1_PA (red)	0x0C	Suggested start: 0x24
LED2_PA (IR)	0x0D	Suggested start: 0x24
REV_ID	0xFE	
PART_ID	0xFF	Reads 0x15

In SpO2 mode each FIFO sample is 6 bytes: red (3 bytes, MSB first) then IR (3 bytes), each with 18 valid bits in the low bits. Samples waiting = $(\text{WR_PTR} - \text{RD_PTR}) \& 0x1F$. At 100 sps, service the sensor every 10 ms (timer tick divided by 10).

9. Interrupt handling contract

- The application's reset code leaves all interrupts masked. Enable them with `irq_enable()` (PicoRV32 `maskirq` with mask 0) only after all peripherals are configured.
- The vector is at `0x1000_0010`. It saves registers, calls the C handler `uint32_t *irq(uint32_t *regs, uint32_t pending)`, restores registers and returns with `retirq`. The existing `start.S` does all of this; do not rewrite it.
- **Interrupts do not nest.** While a handler runs, new interrupts wait.
- **Each handler must check its own device flag**, not just the `pending` argument, and must clear the source before returning. This makes a spurious entry harmless.
- Keep handlers short: copy data into a ring buffer and set a flag. Do UART printing and detector work in the main loop.
- Handler clearing actions: Timer, write 1 to `TIMER_FLAG.TICK`. FIR, write 1 to `FIR_STATUS.DONE`. I2C, write `CR.IACK`.
- Shared variables between handler and main loop must be `volatile`. For ring buffers use a single writer and a single reader with separate indices.

10. Reserved blocks

MI modem (`0x7000_0000` , `irq[6]`): STRETCH. No register map exists yet. Do not write a driver until one is issued.

PCPI `fir.step` custom instruction: STRETCH. Not memory-mapped, and it has no registers. It will be added as an inline-assembly wrapper if it is built.

11. Board differences: Boolean Board or ZedBoard

The project must run on either board. The **register map, addresses, clock rate, bootloader protocol and peripheral behaviour are identical**; only the items below differ. Keep every board-specific value in one `board.h`, selected by `BOARD_BOOLEAN` or `BOARD_ZEDBOARD`. No other file may contain a board name.

Item	Boolean Board (XC7S50)	ZedBoard (XC7Z020, programmable logic only)
System clock	100 MHz (confirm the oscillator)	100 MHz PL oscillator (confirm)
LEDs	16	8
Slide switches	16	8
Push buttons (excluding reset)	4 (confirm)	5
UART to the PC	On-board USB-UART reaches the PL (the bootloader manual uses COM16)	The on-board USB-UART is wired to the processing system, so use a Pmod USB-UART or a 3.3 V USB-TTL adapter on a Pmod pin pair, with common ground
I2C (MAX30102)	Pmod, 3.3 V	Use Pmod JA or JB (3.3 V). Verify the bank voltage of any other Pmod before use
Pin constraints	Per-board XDC	Per-board XDC

`board.h` must provide at least:

```
#define BOARD_LED_COUNT    16    /* Boolean: 16, ZedBoard: 8 */
#define BOARD_SW_COUNT     16    /* Boolean: 16, ZedBoard: 8 */
#define BOARD_BTN_COUNT    4     /* Boolean: 4,  ZedBoard: 5 */
```

Application LED usage stays on bits 0 to 7 (section 5.2), so the same `app.bin` runs on both boards. Only the bitstream, the constraints file and the physical UART connection change.

12. Driver deliverables and acceptance tests

Driver	Functions (suggested)	Acceptance test (print PASS or FAIL over UART)
<code>soc_regs.h</code> , <code>board.h</code>	Address macros, board constants	Compiles; reading <code>0x6000_0000</code> before I2C exists returns <code>0xDEADBEEF</code>
<code>uart</code>	<code>uart_putc</code> , <code>uart_puts</code> , <code>uart_dec</code> , <code>uart_getc_timeout</code>	Echo test: every received byte is sent back; TX polling never loses a character at 4 KB of output
<code>gpio</code>	<code>led_set</code> , <code>led_get</code> , <code>sw_get</code> , <code>btn_get</code>	Walking LED; LEDs mirror the switches
<code>timer</code>	<code>timer_init(hz)</code> , <code>timer_ticks()</code>	Tick count after 10 s of wall-clock time is 10,000 plus or minus 1 at 1 kHz
<code>irq</code> (provided)	<code>irq_enable</code> , <code>irq_disable</code> , dispatcher	Handler runs exactly once per tick; no spurious extra entries over 10 s
<code>fir</code>	<code>fir_init</code> (load + flush), <code>fir_step_poll</code> , <code>fir_start_irq</code> , <code>fir_result</code>	For each channel, 128 samples match <code>sw_fir_step</code> exactly; four channels interleaved in random order also match; a full-scale square wave on channel 3 saturates without mismatch (use a 64-bit reference)
<code>i2c</code>	<code>i2c_init(scl_hz)</code> , <code>i2c_write_reg</code> , <code>i2c_read_reg</code> , <code>i2c_read_burst</code>	Read MAX30102 PART_ID = <code>0x15</code> ; reading a non-existent address (for example <code>0x10</code>) returns an error, not a hang
<code>max30102</code>	<code>max30102_init</code> , <code>max30102_samples_available</code> , <code>max30102_read(&red, &ir)</code>	Samples arrive at 100 per second; red/IR DC level rises when a finger is placed on the sensor

Driver	Functions (suggested)	Acceptance test (print PASS or FAIL over UART)
Application glue	Sample scheduler, <code>det_step</code> , alarm, benchmark	Test vector produces about 75 bpm; benchmark prints cycles per sample for software FIR and hardware FIR

Timing budget (100 MHz)

Task	Rate	Time available	Cost
ECG sample through FIR	250 Hz	4 ms	0.67 μ s for the filter plus bus accesses
PPG sample via I2C burst (about 10 bytes at 100 kHz)	100 Hz	10 ms	about 1 ms
UART line of 30 characters	per beat	n/a	about 2.6 ms, in the main loop only

Rules for all drivers

- Every wait on hardware has a timeout and returns an error. No infinite loops on a status bit.
- No driver may call `uart_*` from interrupt context.
- Every driver exposes an init function that is safe to call twice, because a CPU reset without bitstream reload leaves some hardware state behind.

13. Open items

#	Item	Owner
1	Move the RTL decoder to the address map in section 2.1 and switch the UART to the bootloader's UART	Hardware

#	Item	Owner
2	Enlarge App RAM from 8 KB to 64 KB and confirm the bootloader stack, length header byte order and size limit	Hardware
3	UART: confirm the frame format, whether reading DATA clears RX_VALID, and the TX_BUSY write behaviour	Hardware
4	GPIO: widen LED and switch registers to 16 bits and set the button width	Hardware
5	Integrate the OpenCores I2C with a bridge; confirm the offsets in section 8.1	Hardware
6	Decide whether to clear FIR coefficient memory and delay lines on reset (this spec documents the current behaviour)	Hardware and firmware
7	Update the architecture diagram: MI modem uses <code>irq[6]</code> , I2C uses <code>irq[5]</code>	Architecture
8	Confirm the board and the exact oscillator, LED, switch and button counts once it is issued	Everyone
9	MI modem register map and PCPI wrapper, only if the stretch goals are approved	Hardware

Revision history

Rev	Date	Change
0.1	8 October 2026	First draft, based on the current RTL, the bootloader manual and the architecture update